AMD DEVELOPER HACKATHON ACT III · REINVENT COMMERCE

Chirak

Describe a task, give a few examples, get a small fine-tuned model that does that one job on your own server.

Чирак: apprentice. A large model teaches a small one a single craft.

github.com/escapeboy/chirak · MIT

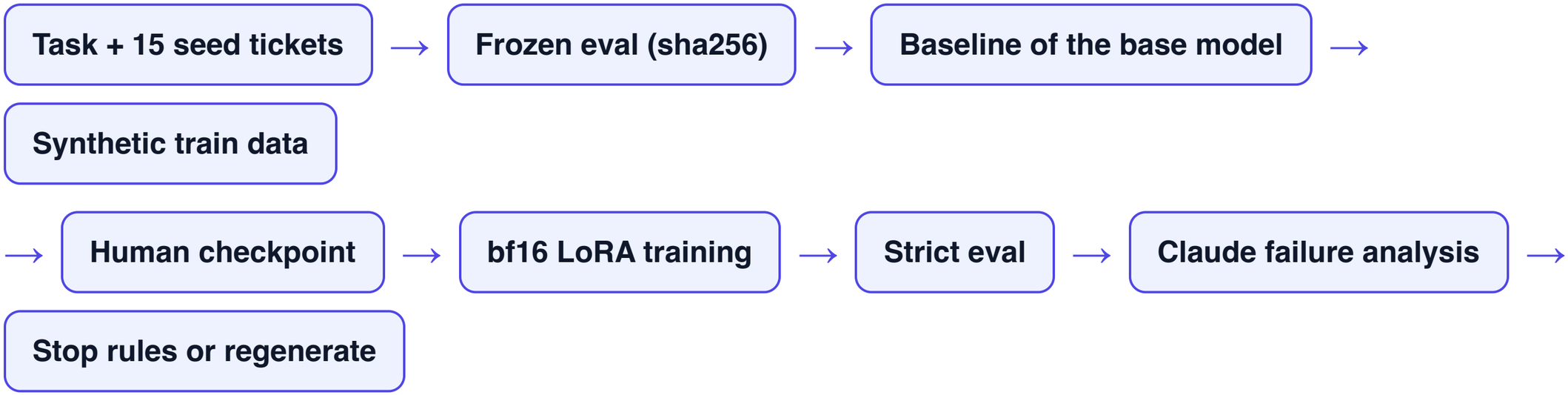
PROBLEM

Shops need a narrow model, not a data science team

- **Narrow, repetitive jobs:** route a support ticket, flag what needs a person, set urgency. A general chatbot API is overkill and costs per call forever.
- **Data stays home:** customer tickets carry names, orders and payment details. In the EU (GDPR) many shops prefer a model on their own server.
- **No ML engineer:** building a fine-tune means data, an honest eval, training and serving. Small merchants cannot staff that.
- **Bulgarian matters:** customers write BG, EN and both in one message.

WHAT IT DOES

One loop, from description to served adapter



The output is a LoRA adapter for Qwen3.5-4B, a public report built from stored results, and a playground that compares the adapter with the base model side by side.

WHY IT IS DIFFERENT

Measured honestly, stopped by rules

Eval first

The eval set is generated and frozen with a sha256 before any training data exists. A reply that is not exactly the JSON object counts as wrong.

No leakage

Train data is written by a different prompt that cannot see eval. Near-copies (bge-m3 cosine ≥ 0.92 or a shared 8-gram) are dropped and counted.

Failure-driven data

Claude reads the confusion matrix and failures and asks for the next slices in strict JSON. Guidance that quotes eval text is discarded.

Deterministic stop

Target met, 3 iterations, under 1 point of gain, or leakage over 30%. In code, not left to the LLM. It also says "not worth fine-tuning".

BEST RESULT · RUN 6 · MERCHANT TICKET TRIAGE, BG/EN

All three fields right: 0.65 → 0.93, target met

RUN 6, EXACT MATCH	SCORE
Base 4B, no fine-tune	0.651
+ LoRA, iteration 1	0.907
+ LoRA, iteration 2 (served)	**0.926**

Run 6 (`merchant-triage-orv1o6`): clearer guidelines (v2) plus 378 real support chats from the ABCD train split in every training step. Target 0.92 met, so the loop stopped. Exact match means category, urgency and needs_human all correct, on run 6's frozen synthetic eval (215 tickets). Its blind label check kept only unambiguous tickets, so this set is easier than real text: see the right-hand tables.

REAL TWEETS, 356, ALL 3 RIGHT	SCORE
Base 4B (run 6 base: 0.35)	0.29 – 0.35
Run 4, synthetic data only	0.43
Run 5b	0.56
Run 6	**0.59**

OWN TICKETS, 146 (RUN 6)	BASE	RUN 6
category	67.8%	71.2%
urgency	47.9%	80.1%
needs_human	76.7%	85.6%

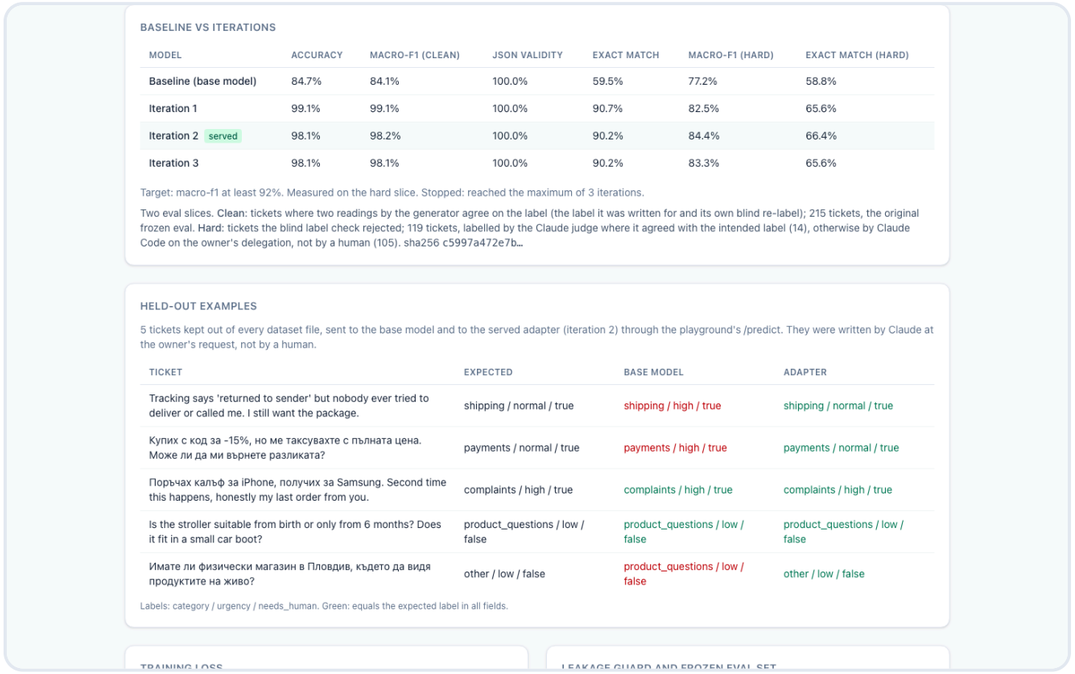
Labels on the real tweets come from Claude, not from people. Live CPU playground: chirak.fleetq.net

THE FIRST STEP · RUN 4 · SYNTHETIC DATA ONLY

Run 4: clean slice 0.84 → 0.98, hard slice 0.77 → 0.84, target not met

RUN 4, MACRO-F1 (CATEGORY)	CLEAN, 215	HARD, 119
Base 4B, no fine-tune	0.84	0.77
+ LoRA, iteration 1	0.99	0.83
+ LoRA, iteration 2 (served)	**0.98**	**0.84**
+ LoRA, iteration 3	0.98	0.83

Clean: tickets the 27B generator labels the same way twice, the clear cases. Hard: tickets that check rejected, labelled by a Claude judge and reviewed by Claude Code on the owner's delegation, not by a human. Target 0.92 on the hard slice was not met; the loop stopped at 3 iterations. Hard slice, all 3 fields right: 0.59 → 0.66. Re-scored with `make eval`: iteration 2 hard 0.8441, same as stored. Run 6 on the previous slide fixed what run 4 left open.



BASELINE VS ITERATIONS

MODEL	ACCURACY	MACRO-F1 (CLEAN)	JSON VALIDITY	EXACT MATCH	MACRO-F1 (HARD)	EXACT MATCH (HARD)
Baseline (base model)	84.7%	84.1%	100.0%	59.5%	77.2%	58.8%
Iteration 1	99.1%	99.1%	100.0%	90.7%	82.5%	65.6%
Iteration 2 served	98.1%	98.2%	100.0%	90.2%	84.4%	66.4%
Iteration 3	98.1%	98.1%	100.0%	90.2%	83.3%	65.6%

Target: macro-f1 at least 92%. Measured on the hard slice. Stopped: reached the maximum of 3 iterations.

Two eval slices. Clean: tickets where two readings by the generator agree on the label (the label it was written for and its own blind re-label); 215 tickets, the original frozen eval. Hard: tickets the blind label check rejected; 119 tickets, labelled by the Claude judge where it agreed with the intended label (14), otherwise by Claude Code on the owner's delegation, not by a human (105). sha256 c5997a472e7b…

HELD-OUT EXAMPLES

5 tickets kept out of every dataset file, sent to the base model and to the served adapter (iteration 2) through the playground's /predict. They were written by Claude at the owner's request, not by a human.

TICKET	EXPECTED	BASE MODEL	ADAPTER
Tracking says 'returned to sender' but nobody ever tried to deliver or called me. I still want the package.	shipping / normal / true	shipping / high / true	shipping / normal / true
Купих с код за -15%, но ме таксувахте с пълната цена. Може ли да ми върнете разликата?	payments / normal / true	payments / high / true	payments / normal / true
Поръчах калъф за iPhone, получих за Samsung. Second time this happens, honestly my last order from you.	complaints / high / true	complaints / high / true	complaints / high / true
Is the stroller suitable from birth or only from 6 months? Does it fit in a small car boot?	product_questions / low / false	product_questions / low / false	product_questions / low / false
Имате ли физически магазин в Пловдив, където да видя продуктите на живо?	other / low / false	product_questions / low / false	other / low / false

Labels: category / urgency / needs_human. Green: equals the expected label in all fields.

TRAINING LOSS

LEAKAGE GUARD AND FROZEN EVAL SET

BUILT ON AMD

Everything runs on one AMD Instinct GPU

STEP	STACK	RUN 4 ON MI350X
Data generation, Qwen3.8-27B	vLLM 0.31 on ROCm 7.2	1000 train items from 3923 candidates, ~6 min
Leakage guard, bge-m3	PyTorch ROCm	same GPU
LoRA training, Qwen3.5-4B bf16	torch 2.14.1+rocm7.2, peft, trl	1000 examples, 2 epochs, 130 s, 12 GB
Eval and serving	vLLM, runtime LoRA hot-load	215 tickets per eval

Iteration 1 took about 17 minutes of GPU time, ≈ $1.60 at $5.49/h; the hard slice and two more iterations about 2.3 h more. The spike passed on MI300X (192 GB); on 13 Oct RunPod had no MI300X left, so the demo ran on an MI350X (gfx950, 288 GB) after re-running the training and serving checks there. No QLoRA, no bitsandbytes.

How it is split: a Laravel 12 control plane (run state machine, stop rules, Claude failure analyst, public reports that work with the GPU off) and a FastAPI GPU worker (/generate, /train, /evaluate, /serve, /predict) reached only through an SSH tunnel. Open source, MIT; tests need no GPU.

MARKET

Who pays first

Online shops

Ticket triage, order-intent detection, review moderation in their own languages. Pay per run: GPU time plus a margin. A run like the demo uses about $1.60 of GPU.

Regulated sectors

Banks, health and public bodies that cannot send texts to an external API. Self-hosted licence: the factory and the adapters run inside their network.

Agencies

Integrators who build support tooling for many small merchants and need a repeatable way to produce and prove a small model per client.

This slide is a plan, not traction: there are no customers yet.

NEXT

Next steps

- **Human-labelled evals:** the hard slice, the real-tweet set and the own-ticket flow exist, but their labels come from a Claude judge and Claude Code; Bulgarian tickets labelled by people are next.
- **Extraction tasks:** order numbers, products, amounts, not only classification.
- **Multi-tenant runs** on a shared GPU with a queue, and adapters per merchant.
- **Preference tuning (DPO)** once a classification adapter exists.
- **A marketplace of adapters** for common commerce tasks, each with its frozen eval and report.